

PURBANCHAL UNIVERSITY



**DEPARTMENT OF COMPUTER ENGINEERING
KHWOPA ENGINEERING COLLEGE
LIBALI-08, BHAKTAPUR**

**A MID-TERM REPORT
ON
GYM MANAGEMENT SYSTEM**

*Project work submitted in partial fulfillment of requirements for the award of the degree of Bachelor of
Engineering in Computer Engineering (Fifth Semester)*

SUBMITTED BY
Ayush Pokharel (800309)
Bibek G.C (800310)
Bikram Kayastha (800312)
Romis Khatiwada (800333)

UNDER THE SUPERVISION OF
Ms. Ijana Kumpakha

29 May 2026

ACKNOWLEDGEMENT

We express our deep appreciation and sincere gratitude to everyone who played a vital role in the successful completion of this **Gym Management System (GMS)** project. First, we would like to thank **Purbanchal University** and **Khwopa Engineering College, Department of Computer Engineering**, for providing the academic resources and a platform that encourages innovation in modern web-based application development.

Our heartfelt gratitude goes to our project supervisor, **Ms. Ijana Kumpakha**, for her invaluable guidance, technical wisdom, and unwavering support throughout this journey. Her expertise in software engineering and full-stack web technologies was pivotal in shaping the direction of our work, especially in architecting a robust and scalable solution using the **MERN Stack (MongoDB, Express.js, React.js, and Node.js)**.

We are also immensely thankful to **Er. Bikash Chawal** (Head of Department) and all the faculty members of the Department of Computer Engineering for providing the necessary facilities and opportunities that enabled us to execute this project effectively.

In addition, we acknowledge the dedication and teamwork of each member involved in the development phase. The collaborative effort to design the **NoSQL database schema**, implement **RESTful API endpoints**, integrate **Stripe payment processing**, and create a responsive **React-based user interface** has been a significant learning experience for all of us.

Lastly, we extend our thanks to our families and friends for their constant support and motivation, which encouraged us throughout this project. This system is the result of a strong collective effort, and we truly appreciate everyone who played a part in building a more **efficient, automated, and streamlined fitness management system**.

Team Members:

- **Ayush Pokharel** (800309)
- **Bibek G.C** (800310)
- **Bikram Kayastha** (800312)
- **Romis Khatiwada** (800333)

ABSTRACT

The **Gym Management System (GMS)** is designed to automate and streamline the management of fitness center operations through a centralized, modern web-based platform. Moving away from traditional manual record-keeping and standalone console applications, this system provides administrators with a robust digital environment to oversee complex gym activities efficiently. The system is built using the **MERN Stack**, leveraging **Node.js** and **Express.js** for high-performance server-side logic and a **MongoDB** NoSQL database to ensure flexible data modeling, reliability, and cloud-based persistence.

GMS offers integrated modules such as member registration, trainer specialization management, membership plan tracking, and automated digital billing. By utilizing a component-based frontend approach with **React.js**, along with **HTML5** and **CSS3**, the system ensures a highly responsive and fluid user interface accessible across multiple devices. To ensure data privacy and security, the application is fortified with **JSON Web Token (JWT)** authentication and **BcryptJS** password encryption, providing controlled and secure access for administrators and staff. Furthermore, the system integrates the **Stripe API** for secure online payments and **Cloudinary** for cloud-hosted media management. The expected outcome of this project is a scalable, full-stack management tool that simplifies data retrieval and real-time reporting, ultimately enhancing overall operational productivity and accuracy within the fitness industry.

Table of Contents

Cover Page	i
Acknowledgement	ii
Abstract	iii
Table of Contents	v
List of Figures	vi
List of Tables	vii
List of Abbreviations	viii
1 INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Statement of Problems	2
1.4 Objectives	2
1.5 Scope and Application	2
2 LITERATURE REVIEW	4
3 PROJECT MANAGEMENT	6
3.1 Team Members	6
3.2 Feasibility Study	6
3.2.1 Economic Feasibility	6
3.2.2 Technical Feasibility	6
3.2.3 Schedule Feasibility	7
3.2.4 Operational Feasibility	7
3.3 Work Breakdown Structure (WBS)	8
4 METHODOLOGY	9
4.1 Background	9
4.2 Block Diagram	10
4.3 Use Case Diagram	11
4.4 Algorithm	11
4.5 ER Diagram	14
4.6 Tools and Platforms	14

5	ACHIEVED OUTCOMES	16
5.1	Automated Gym Management	16
5.2	User-Friendly Interface	16
5.3	Secure Authentication and Data Management	18
5.4	Integrated Payment System	19
5.5	Membership and Class Enrollment	19
5.6	Trainer Portal	20
5.7	Efficient Record-Keeping and Scalable Database	20
5.8	Role-Based Access Control	21
5.9	Class Reviews and Ratings	21
6	REFERENCES	22

List of Figures

4.1	Block Diagram	10
4.2	Use Case Diagram	11
4.3	ER Diagram	14
5.1	Admin Dashboard – Centralized Gym Management	16
5.2	Home Page – Class Browsing Interface	17
5.3	Secure Login Interface for Authenticated Access	18
5.4	New Member Registration and Account Creation	18
5.5	Stripe Payment Integration – Membership Checkout	19
5.6	Multi-Step Membership Enrollment Flow	19
5.7	Trainer Portal – Class and Trainee Management	20
5.8	Membership Records – Admin View	20
5.9	Role-Based Dashboards – Admin, Trainer, and Trainee Views	21
5.10	Class Reviews and Ratings System	21

List of Tables

3.1	Project Timeline	8
-----	----------------------------	---

List of Abbreviations

API	Application Programming Interface
BMI	Body Mass Index
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
ES6	ECMAScript 6
GMS	Gym Management System
HTML	HyperText Markup Language
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
NoSQL	Non-Relational Database
ODM	Object Document Mapper (Mongoose)
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDLC	Software Development Life Cycle
UI	User Interface
UX	User Experience

Chapter 1

INTRODUCTION

1.1 Background

Gyms play a critical role in promoting health, fitness, and overall well-being among individuals. Delivering high-quality fitness services relies not only on skilled trainers and modern facilities but also on effective, digitized record-keeping. In the modern fitness landscape, accurate and real-time member records are essential for tracking progress, personalizing fitness plans, and ensuring member satisfaction. Relying on manual or standalone console-based systems can lead to suboptimal training experiences, data loss, or missed fitness goals.

The **GMS** is an integrated, full-stack web solution designed to streamline various aspects of gym workflows using the **MERN** stack. Moving beyond the limitations of legacy software, this system enables seamless access to workout plans, class schedules, and progress tracking via a browser-based interface. **GMS** ensures smooth operations by integrating administrative, membership, financial, and training functions into a single, cloud-ready foundation. By leveraging a high-performance NoSQL database and a component-based frontend, the system allows for faster data retrieval and more efficient gym management.

1.2 Motivation

The motivation behind choosing the Gym Management System (GMS) project is rooted in the increasing necessity for fitness centers to adopt scalable, cloud-native solutions that enhance operational efficiency. In the contemporary fitness landscape, manual record-keeping is no longer sustainable due to the high volume of data and the demand for real-time progress updates. By developing a modern web application, we aim to eliminate the delays and errors associated with physical logs or legacy desktop software, providing a high-speed, professional environment for both trainers and members.

A key motivator for this specific implementation is the selection of the **MERN Stack** as the primary development framework. **Node.js** and **Express.js** offer a non-blocking, event-driven architecture that is ideal for handling concurrent user requests in a busy gym environment. The use of **MongoDB** provides a flexible NoSQL schema, allowing the system to store diverse member data without the rigidity of traditional relational tables. Furthermore, the integration of the **Stripe API** for secure digital payments and **Cloudinary** for cloud-hosted media management demonstrates a commitment to building a production-ready system. By leveraging **JavaScript** across the entire stack, we ensure a unified development workflow, resulting in a system that is not only user-friendly but also technically resilient and highly scalable.

1.3 Statement of Problems

- **Data Fragmentation and Retrieval:** In high-traffic gym environments, retrieving specific member information such as workout history or health metrics from physical files or disconnected desktop applications leads to significant operational delays.
- **Lack of Real-Time Cloud Storage:** Many fitness centers lack the capability for real-time data synchronization across devices, making it difficult for trainers to update schedules or attendance logs instantly.
- **Inefficient Updating of Membership Status:** Manually tracking membership expirations and payment statuses is a cumbersome process that often results in financial inaccuracies and delayed renewals.
- **Security and Data Privacy:** Legacy systems often lack robust encryption, leaving sensitive member information and payment details vulnerable to unauthorized access.
- **Administrative Overload:** Reliance on manual paperwork for trainer assignments and class scheduling reduces staff productivity and increases the risk of human error in financial billing and fitness progress metrics.

1.4 Objectives

- To design and develop a comprehensive **MERN-based Gym Management System** that automates member registration, fitness tracking, administrative operations, secure authentication using **JWT**, online membership payments through **Stripe Payment Gateway**, cloud-based media management with **Cloudinary**, and a responsive **React.js** frontend that ensures a seamless user experience across both mobile and desktop platforms.

1.5 Scope and Application

- **Full-Stack Digitalization:** The system provides a highly intuitive, **React-based** interface that eliminates the data loss associated with manual record-keeping and outdated terminal-based software.
- **NoSQL Data Management:** Using **MongoDB**, the system offers a centralized, cloud-hosted repository for member and staff profiles, enabling flexible data handling for workout routines and diet plans.
- **Automated Financial Management:** It streamlines billing through the integration of digital payment APIs, allowing members to subscribe to plans and receive instant confirmation.
- **Enhanced Communication:** Features such as **Nodemailer** integration allow for automated email notifications regarding membership status and gym updates.

- **Scalable Application:** The system is designed for a wide range of environments, from boutique fitness studios to large-scale commercial chains, looking to professionalize their operations and provide members with real-time access to their fitness data.

Chapter 2

LITERATURE REVIEW

The global fitness industry has witnessed an unprecedented surge in recent years, driven by a growing societal emphasis on health, wellness, and preventive medicine. As individuals become increasingly conscious of physical fitness, the demand for well-equipped gym facilities and professional training services has scaled exponentially. However, the operational success of a modern fitness center depends on more than just high-quality equipment and skilled instructors; it is fundamentally tied to the efficiency of its administrative and data management systems. [1]

Historically, gym management was a labor-intensive process characterized by manual record-keeping. Membership enrollments, attendance logs, and payment histories were typically recorded in physical registers or basic ledger books. This “Paper-Based” approach, while simple, presented significant challenges in terms of data integrity, physical storage space, and the sheer time required to retrieve specific information. Furthermore, manual systems were highly susceptible to human error, resulting in inaccurate billing, lost member records, and an inability to track long-term fitness progress effectively. [2]

With the initial advent of computing in small businesses, many gyms migrated toward basic digital solutions such as spreadsheets or standalone desktop applications (often built in languages like C++ or Python). While these tools provided better data storage than paper, they created “Data Silos”—information was locked on a single computer and could not be accessed remotely by trainers or members. These systems lacked real-time synchronization, making it impossible to provide members with instant updates on their subscription status or workout schedules through their own devices. [2]

The current era of gym management is defined by the transition to full-stack web architectures, specifically the **MERN** stack. This shift is motivated by the need for high-speed, concurrent data processing and universal accessibility. Unlike traditional relational databases that require fixed schemas, the use of **MongoDB** in this project allows for a flexible NoSQL approach, accommodating the diverse and evolving data types found in fitness management, such as varied workout metrics, medical histories, and complex nutritional plans. [3, 4]

Modern gym management also necessitates the integration of secure financial and media services. The selection of **Node.js** for the backend provides the event-driven architecture needed to handle a high volume of simultaneous user requests, such as real-time attendance tracking and secure transaction processing via the **Stripe API** [6]. Additionally, the requirement for high-resolution media management led to the implementation of **Cloudinary**, ensuring that member profile data is hosted in a scalable cloud environment [6]. Security across all these layers is enforced through modern authentication mechanisms such as **JWT** and **BcryptJS** [5].

Ultimately, this background study highlights that the development of a **GMS** is not merely a task of digitizing records, but an effort to create a holistic, secure, and responsive ecosystem. By

leveraging a unified JavaScript-based stack, the system aims to solve the historical problems of data fragmentation and administrative overload, providing a professional foundation upon which modern fitness centers can grow and provide superior service to their members. [7]

Chapter 3

PROJECT MANAGEMENT

3.1 Team Members

The project efforts is shared among the following team members:

- Ayush Pokharel (800309)
- Bibek G.C (800310)
- Bikram Kayastha (800312)
- Romis Khatiwada (800333)

3.2 Feasibility Study

A preliminary study was conducted to evaluate the feasibility of the proposed **GMS** within the context of a modern web application framework. The project was analyzed across four critical dimensions: economic, technical, schedule, and operational feasibility. This comprehensive assessment ensures that the project can be successfully developed, deployed, and maintained within the constraints of the current academic and technical environment.

3.2.1 Economic Feasibility

The proposed **MERN-stack** based system is highly feasible from an economic perspective, as it requires minimal financial investment while offering high potential returns in operational efficiency. The primary development tools including **Node.js**, **React.js**, and the **VS Code** editor are open-source and freely available to the public. Furthermore, the database requirements are met using the **MongoDB Atlas** free tier, and media hosting is handled via the **Cloudinary** free tier, eliminating the immediate need for expensive server hardware or cloud storage subscriptions.

Existing computer hardware available to the team is sufficient for the development, local hosting, and testing phases of the project. By automating labor-intensive gym management tasks such as member registration, secure payment handling via **Stripe**, and digital record maintenance, the system significantly reduces the need for physical stationery and manual administrative labor. Consequently, the project is economically feasible as it minimizes initial capital expenditure while providing substantial long-term benefits in terms of resource optimization and error reduction.

3.2.2 Technical Feasibility

The technical feasibility of the Gym Management System is supported by the adoption of industry-standard technologies that are well-documented and widely supported. The selection of the **MERN stack** allows for a “JavaScript Everywhere” approach, which simplifies the development process by using a single language for both the frontend and backend. **Node.js** provides the necessary

non-blocking I/O environment to handle multiple concurrent user sessions, while **React.js** ensures a highly responsive and modular user interface.

The integration of advanced third-party APIs, such as **Stripe** for financial transactions and **Cloudinary** for media hosting, demonstrates the technical maturity of the project. These services provide pre-built, secure infrastructures for complex tasks, allowing the development team to focus on business logic rather than building payment gateways or image processors from scratch. The technical skills required including **RESTful API** development, **NoSQL** data modeling with **Mongoose**, and state management in React are aligned with the core competencies of the computer engineering curriculum. Therefore, the project is technically feasible, as the necessary tools and technical expertise are within the team's reach.

3.2.3 Schedule Feasibility

The project development is structured into a logical sequence of phases designed to ensure timely completion within the academic semester. The **SDLC** for this project includes requirement gathering, system architecture design, backend API development, frontend component creation, and rigorous system testing. Each of these phases is assigned specific deadlines to prevent any single task from becoming a bottleneck.

By utilizing a modular approach where the backend and frontend are developed as independent but interconnected units the team can work on different components simultaneously. The use of **Git** for version control further facilitates collaborative work and helps in tracking progress against the project timeline. Given the current rate of development and the clear roadmap for implementing features like **JWT** authentication and **Stripe** payments, the project is feasible within the available schedule and can be completed by the designated submission date.

3.2.4 Operational Feasibility

Operationally, the Gym Management System is designed to be highly intuitive and user-centric, catering to administrators, trainers, and members with varying levels of technical proficiency. As a browser-based application, it requires no complex installation on the user's end, making it accessible from any device with an internet connection. This "Anywhere Access" significantly improves the operational flexibility of the gym, allowing administrators to monitor attendance and revenue in real-time from any location.

The system incorporates RBAC through **JWT**, ensuring that each user only interacts with the data relevant to their role. This simplifies the workflow for trainers while protecting the integrity of financial and administrative data. By automating tedious processes such as subscription tracking, attendance logging, and billing, the GMS reduces the administrative burden on gym staff and enhances the overall member experience. Thus, the project is operationally feasible as it provides a practical, secure, and easy-to-use solution that directly addresses the real-world needs of fitness facilities.

3.3 Work Breakdown Structure (WBS)

S.N	Task Description	Duration	1st Month				2nd Month				3rd Month			
			1st	2nd	3rd	4th	1st	2nd	3rd	4th	1st	2nd	3rd	4th
1	Problem Identification	3 weeks												
2	Analysis	4 weeks												
3	Design	5 weeks												
4	Coding	6 weeks												
5	Implementing and Testing	3 weeks												
6	Documentation	12 weeks												

Table 3.1: Project Timeline

Chapter 4

METHODOLOGY

4.1 Background

The development of the **GMS** is grounded in the principles of modern full-stack web engineering. Historically, management systems were developed as monolithic applications where the user interface and data logic were tightly coupled, often leading to challenges in scalability and maintenance. This project, however, adopts the **Decoupled Architecture** provided by the **MERN** stack. The background of this methodological choice lies in the need for a unified, high-performance environment where a single language **JavaScript** can be utilized across the entire development spectrum. This “Single Language” approach reduces the complexity of context-switching between different programming languages and ensures a more seamless integration between the frontend and backend layers.

A significant factor in the background of this methodology is the transition from **Relational (SQL)** to **Document-Oriented (NoSQL)** data modeling. In a fitness management context, data is often non-uniform; for instance, different members may have vastly different health metrics, workout histories, and subscription details. Traditional SQL databases require a fixed schema that can be restrictive when the system needs to evolve. The choice of **MongoDB** provides the methodological flexibility to store data as JSON-like documents. This allows the system to be “schema-less” at the database level while maintaining data integrity at the application level through **Mongoose** models. This background logic ensures that the system is not only robust for current requirements but also adaptable to future data expansion.

Furthermore, the methodology is influenced by the shift toward **Stateless Authentication** and **API-first development**. In older systems, user sessions were often stored on the server’s memory, which limited the system’s ability to scale across multiple servers. The methodology of this project is built upon the background of **JWT**. By utilizing JWTs, the authentication logic becomes stateless and decentralized, allowing the **React.js** frontend to interact with the **Node.js** backend via secure, independent **RESTful API** calls. This architectural background ensures that the system remains responsive and secure, even as the number of concurrent users grows.

Lastly, the methodological background emphasizes the integration of **Third-Party Cloud Services** as a core strategy for system reliability. Rather than building complex financial modules or image processing engines from scratch, the methodology leverages the proven infrastructures of **Stripe** for payment processing and **Cloudinary** for media hosting. This approach is rooted in the “Don’t Reinvent the Wheel” philosophy of software engineering, which allows the development team to focus on the unique business logic of gym management while relying on industry-standard APIs for specialized tasks. This background ensures that the resulting system is production-ready, highly secure, and adheres to the best practices of modern web development.

4.2 Block Diagram

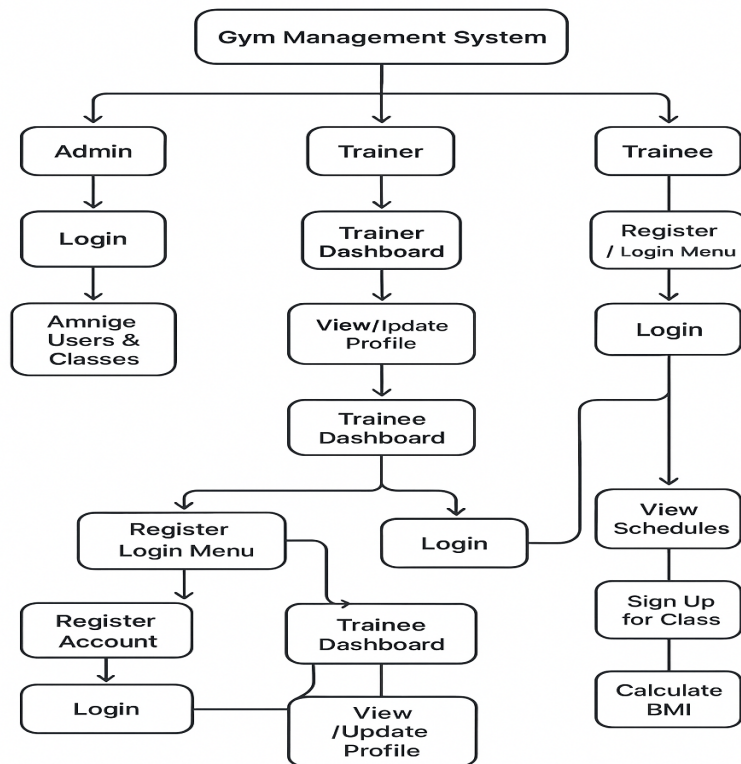


Figure 4.1: Block Diagram

4.3 Use Case Diagram

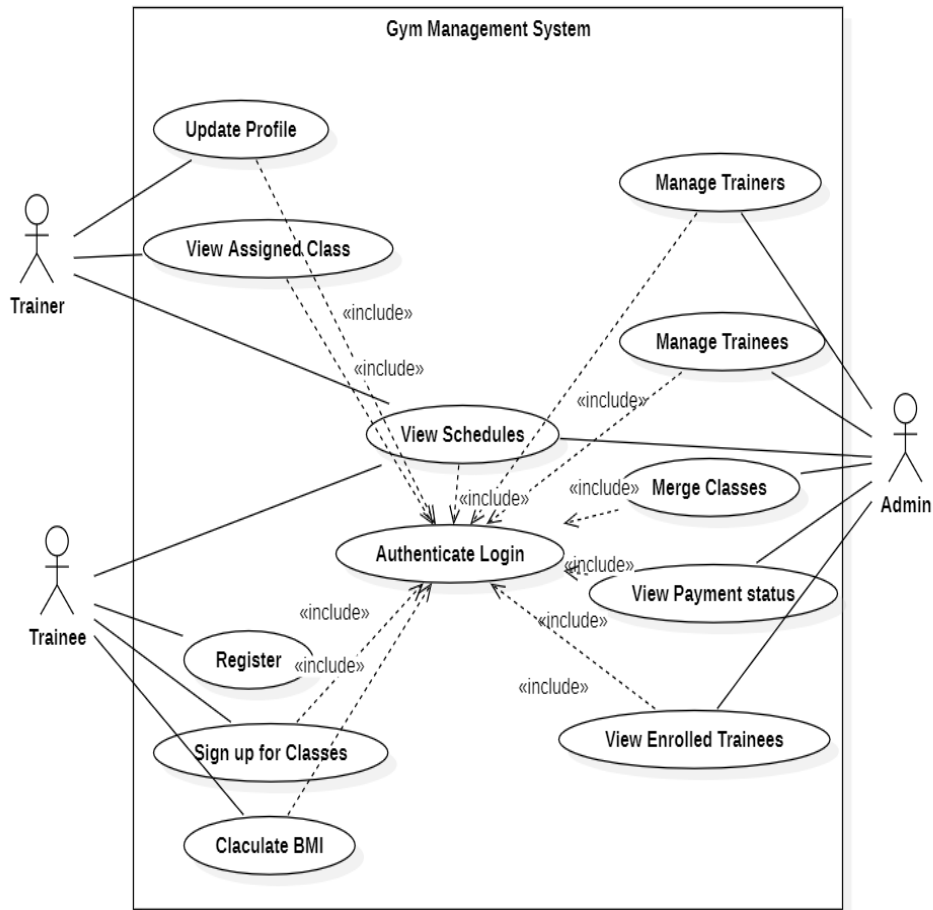


Figure 4.2: Use Case Diagram

4.4 Algorithm

The following algorithm outlines the logical flow of the web-based Gym Management System using the MERN stack architecture.

Step 1: Initialization: Start the application by launching the React frontend and connecting the Node.js backend to the MongoDB Atlas cloud database.

Step 2: User Entry: The user accesses the landing page. The system checks for a valid **JWT** in the browser's cookies or local storage.

- If a valid token exists, redirect the user to their respective Dashboard based on their **Role** (Admin, Trainer, or Member).

- If no token exists, display the Login/Registration interface.

Step 3: Authentication Process (Login/Register):

- User submits credentials (email and password) via a **POST request** to the Express backend.
- **Backend Validation:** The system retrieves the user document from MongoDB.
- **Password Verification:** The system compares the entered password with the stored hash using the **BcryptJS** library.
- **Token Generation:** If valid, the backend generates a **JWT** containing the user's ID and Role, and sends it back to the client.

Step 4: Role-Based Routing: The React Router intercepts the authenticated state and grants access to protected routes:

- a) **Admin:** Access to the Management Console.
- b) **Trainer:** Access to the Training Portal.
- c) **Member:** Access to the Personal Fitness Profile.

Step 5: Admin Operations (Management Hub):

- **CRUD Operations:** Admin can Add, View, Update, or Delete Members and Trainers.
- **Media Management:** When an image is uploaded (e.g., Trainer profile), the system sends the file to the **Cloudinary API** and stores the returned URL in MongoDB.
- **System Overview:** Fetch real-time counts of active subscriptions and total revenue from the database.

Step 6: Trainer Operations (Training Portal):

- **Data Retrieval:** Fetch assigned member lists via specialized API endpoints.
- **Progress Tracking:** Update workout and diet schemas for specific members.

Step 7: Member Operations (User Portal):

- **Profile Access:** View personalized workout schedules and health metrics (BMI).
- **Subscription Payment:**
 - i. Member selects a plan and initiates the payment process.
 - ii. The system creates a **Stripe Payment Intent**.
 - iii. Upon successful card verification by **Stripe**, the backend updates the member's subscription status in MongoDB.
 - iv. Generate a digital receipt and send an email confirmation via **Nodemailer**.

Step 8: Data Persistence: Any modification made during the session is sent as a **PUT** or **POST** request to the Node server, ensuring the MongoDB Atlas cloud storage is updated asynchronously without refreshing the page.

Step 9: Session Termination (Logout):

- The user clicks “Logout.”
- The system destroys the JWT token on the client-side and clears the browser cookies.
- Redirect the user back to the Login page.

Step 10: End: Terminate the current user session.

4.5 ER Diagram

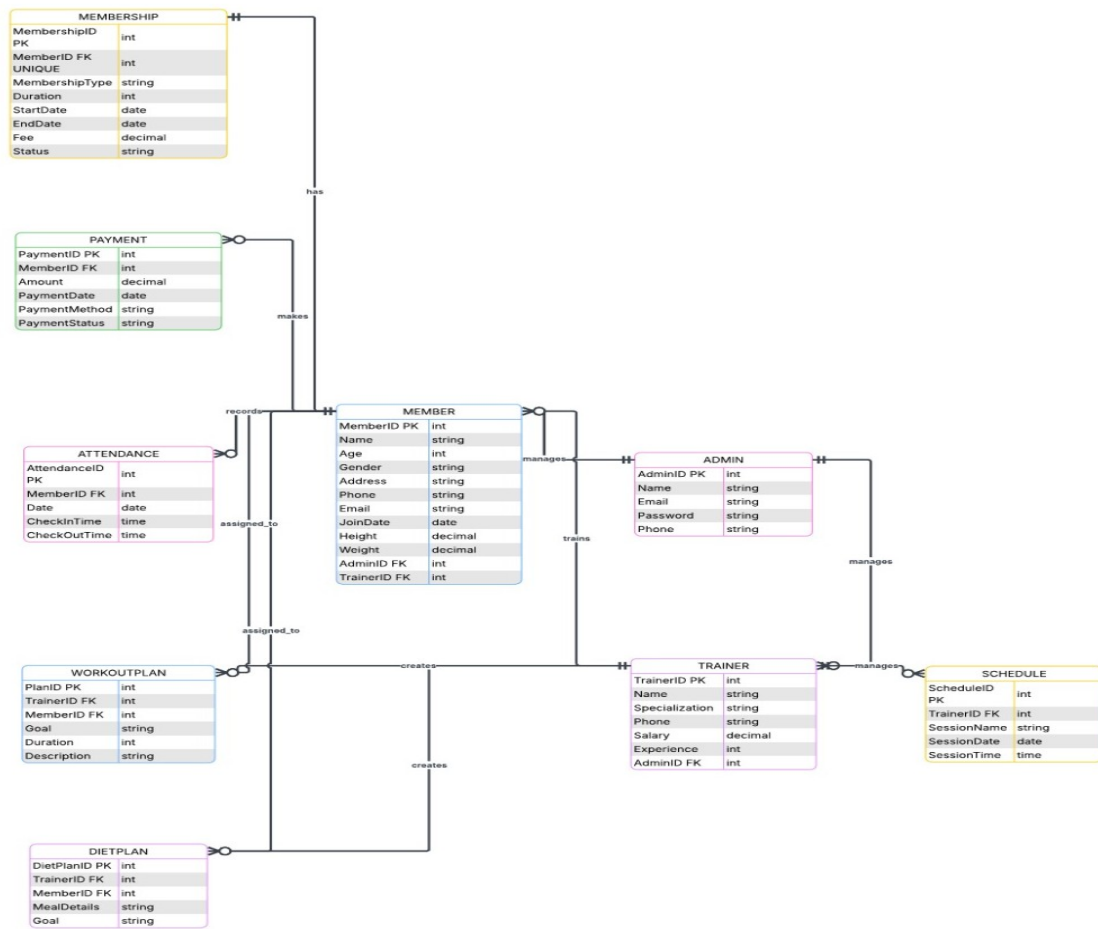


Figure 4.3: ER Diagram

4.6 Tools and Platforms

The development and deployment of the Gym Management System utilize the following:

- **Programming Languages:** JavaScript (ES6+), HTML5, CSS3
- **Frontend Library:** React.js (Component-based UI development)
- **Backend Environment:** Node.js (Runtime environment)
- **Backend Framework:** Express.js (Rest API development)
- **Database System:** MongoDB Atlas (Cloud-based NoSQL database)
- **Data Modeling:** Mongoose (Object Data Modeling for Node.js)

- **Security & Authentication:** JSON Web Tokens (JWT), Bcryptjs (Password Hashing)
- **Third-Party APIs:**
 - **Stripe API:** For secure online membership payments.
 - **Cloudinary API:** For cloud-hosted image and media management.
 - **Nodemailer:** For automated email notifications and receipts.
- **IDE:** Visual Studio Code
- **Development Tools:** Postman (API Testing), Git, GitHub
- **Operating Systems:** Windows, macOS, Linux

Chapter 5

ACHIEVED OUTCOMES

Upon the successful development and implementation of the Gym Management System (GMS) using the MERN Stack, the following outcomes have been achieved:

5.1 Automated Gym Management

The system automates core gym operations including membership enrollment, class scheduling, and billing. Administrators can manage the entire gym workflow from a centralized dashboard without manual intervention.

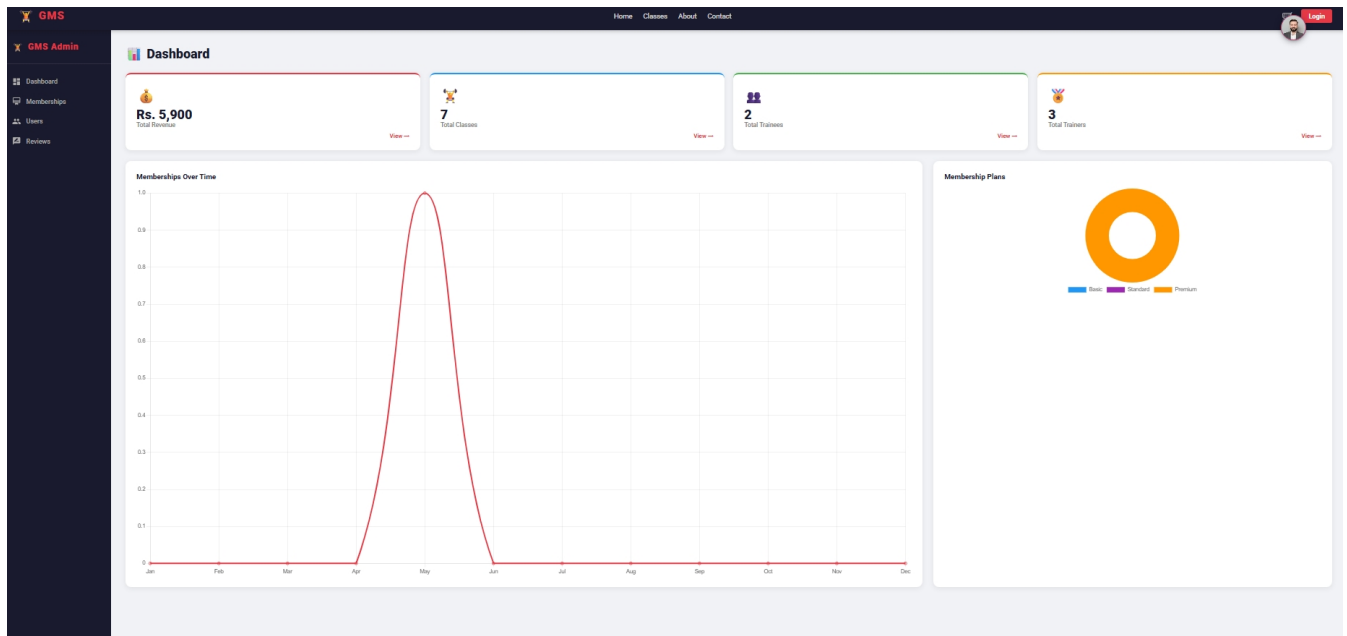


Figure 5.1: Admin Dashboard – Centralized Gym Management

5.2 User-Friendly Interface

A fully responsive and intuitive React.js-based user interface has been developed, ensuring a seamless experience across all user roles – admins, trainers, and trainees. The interface includes a structured navigation header, class browsing pages, and dedicated user profile sections.

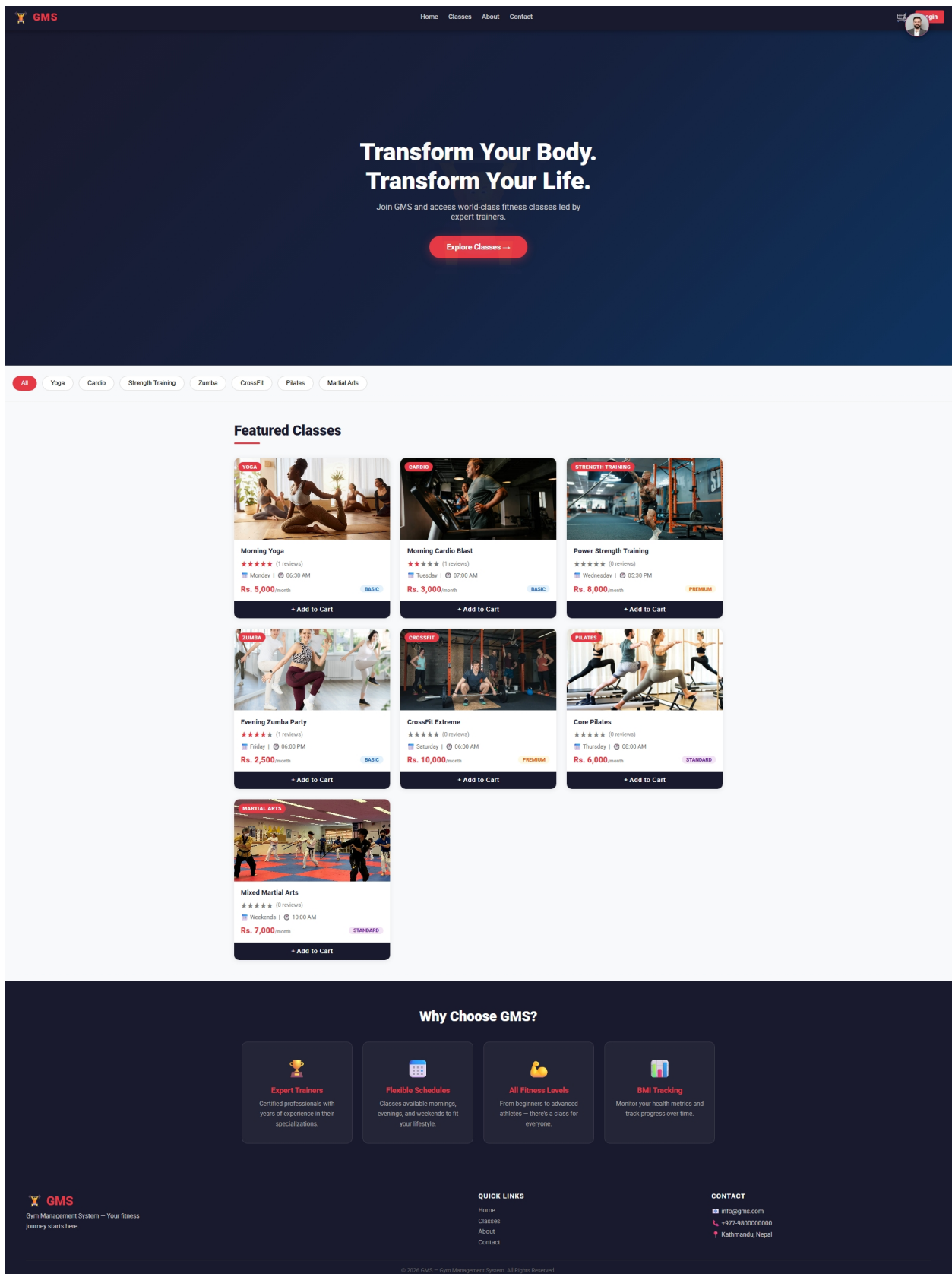
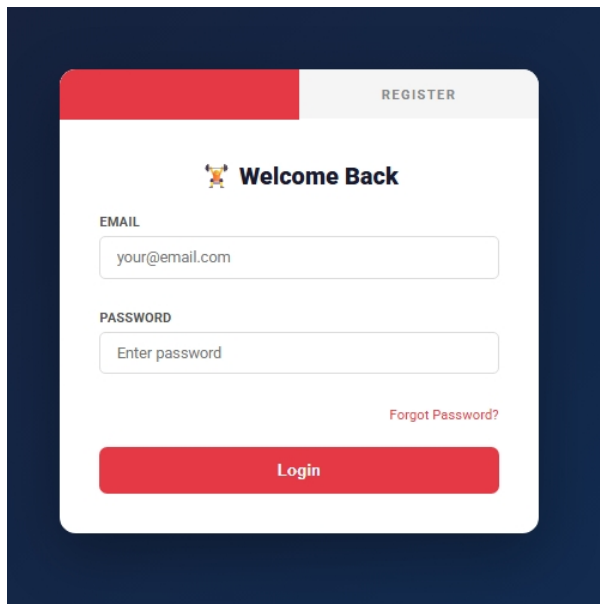


Figure 5.2: Home Page – Class Browsing Interface

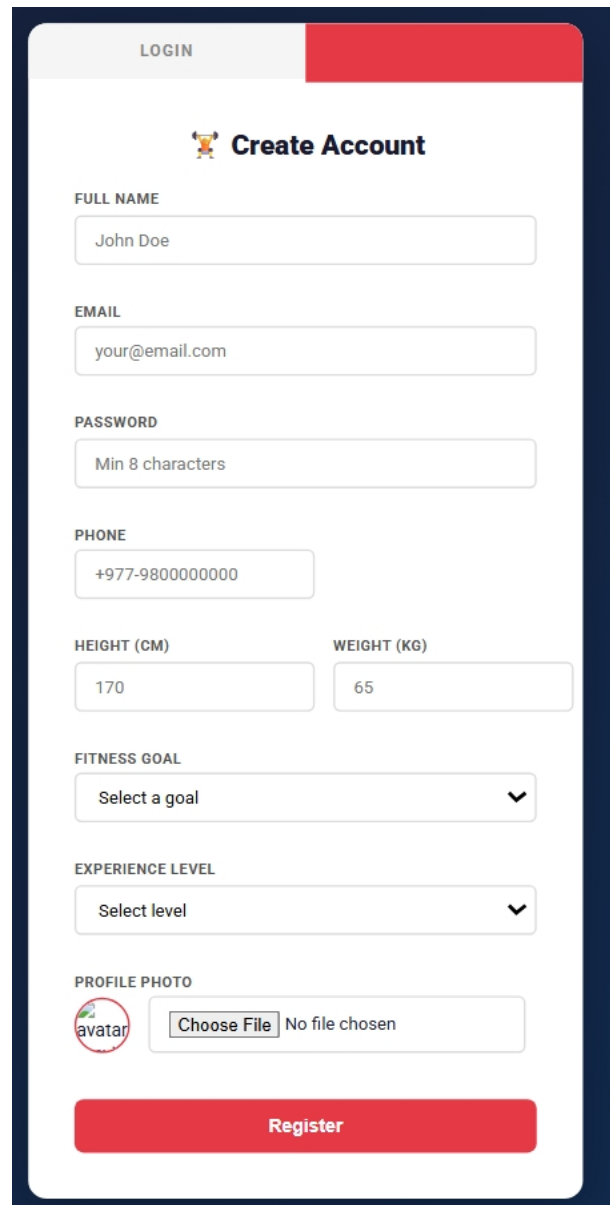
5.3 Secure Authentication and Data Management

The system implements **JWT-based authentication**, **bcrypt password hashing**, and **role-based access control (RBAC)** to ensure that sensitive member health and financial data remains protected. Password reset functionality via secure email tokens has also been integrated to maintain account recovery standards.



The login interface features a dark blue background with a white login card. At the top, a red header bar contains the word "REGISTER" in white. Below the header, a "Welcome Back" message is displayed with a small icon. The form includes fields for "EMAIL" (containing "your@email.com") and "PASSWORD" (containing "Enter password"). A red "Login" button is positioned at the bottom. A "Forgot Password?" link is located below the password field.

Figure 5.3: Secure Login Interface for Authenticated Access



The registration interface features a dark blue background with a white registration card. At the top, a red header bar contains the word "LOGIN" in white. Below the header, a "Create Account" message is displayed with a small icon. The form includes fields for "FULL NAME" (containing "John Doe"), "EMAIL" (containing "your@email.com"), "PASSWORD" (containing "Min 8 characters"), and "PHONE" (containing "+977-9800000000"). There are also fields for "HEIGHT (CM)" (containing "170") and "WEIGHT (KG)" (containing "65"). Below these are dropdown menus for "FITNESS GOAL" (containing "Select a goal") and "EXPERIENCE LEVEL" (containing "Select level"). At the bottom, there is a "PROFILE PHOTO" section with a circular avatar placeholder and a "Choose File" button. A red "Register" button is positioned at the bottom.

Figure 5.4: New Member Registration and Account Creation

5.4 Integrated Payment System

Stripe payment gateway has been successfully integrated into the membership checkout flow, enabling secure and reliable online payment processing for membership fees.

The screenshot shows a web application interface for a gym membership checkout. At the top, there is a dark navigation bar with the 'GMS' logo and links for Home, Classes, About, and Contact. A user profile icon is visible in the top right corner. Below the navigation bar, a progress indicator shows three steps: 1. Health Info, 2. Confirm Membership, and 3. Payment. The main content area displays a 'Complete Payment' form. The form includes a Stripe logo, the title 'Complete Payment', and the total amount 'Total: Rs. 9,440'. It contains input fields for 'CARD NUMBER' (4242 4242 4242 4242), 'EXPIRY DATE' (12 / 26), and 'CVC' (567). A 'Save with link' button is next to the card number field. At the bottom of the form is a large red button labeled 'Pay Rs. 9,440'.

Figure 5.5: Stripe Payment Integration – Membership Checkout

5.5 Membership and Class Enrollment

Members can browse available gym classes, add them to cart, provide health information, confirm membership plans (Basic, Standard, Premium), and complete enrollment in a structured multi-step checkout process.

The screenshot displays a multi-step membership enrollment flow. At the top, a progress bar indicates three steps: 1. Health Info, 2. Confirm Membership, and 3. Payment. The main content area is divided into three sections. The first section, 'Health Info', contains a form with fields for HEIGHT (170 cm), GOAL (General Fitness), PHONE (+977 9844789283), DURATION (3 Months), WEIGHT (58 kg), LEVEL (Beginner), and PLAN (Standard). The second section, 'Selected Classes', shows a list of classes with a thumbnail for 'Power Strength Training' and details: '1 session/week × Rs. 8,000'. The third section, 'Membership Summary', provides a breakdown of costs: Classes Price (Rs. 8,000), Facility Fee (18%) (Rs. 1,440), and Processing Fee (Free), resulting in a Total of Rs. 9,440. A red button labeled 'Proceed to Payment →' is located at the bottom of the summary section.

Figure 5.6: Multi-Step Membership Enrollment Flow

5.6 Trainer Portal

A dedicated trainer portal has been implemented, allowing trainers to view their assigned classes and manage trainee information. Access is restricted exclusively to users with the trainer or admin role.

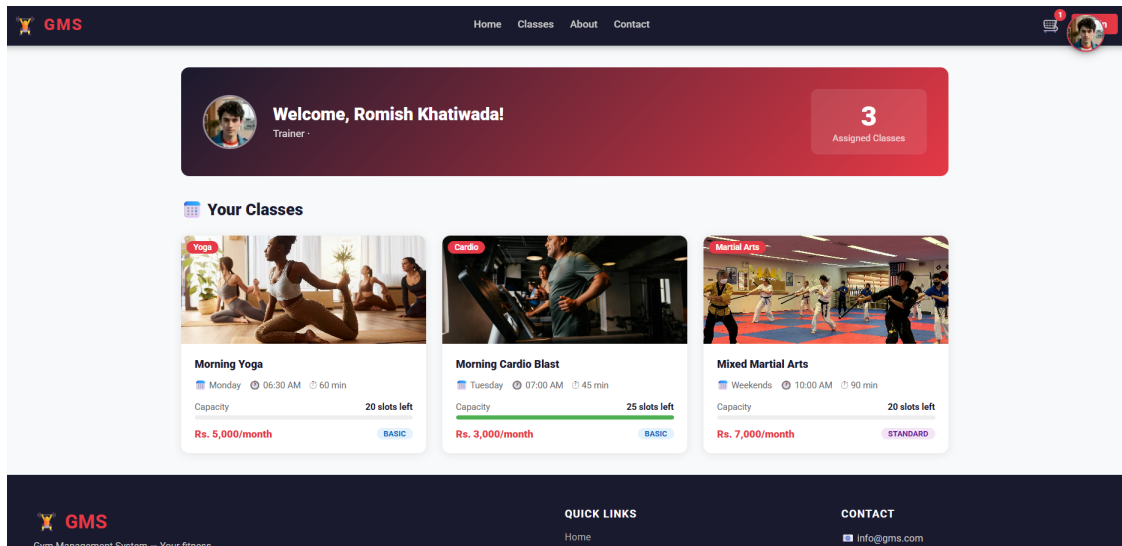


Figure 5.7: Trainer Portal – Class and Trainee Management

5.7 Efficient Record-Keeping and Scalable Database

MongoDB has been utilized as the primary database, enabling organized and scalable storage of user profiles, class records, membership data, and payment history. The NoSQL schema supports fast and reliable data retrieval across all modules.

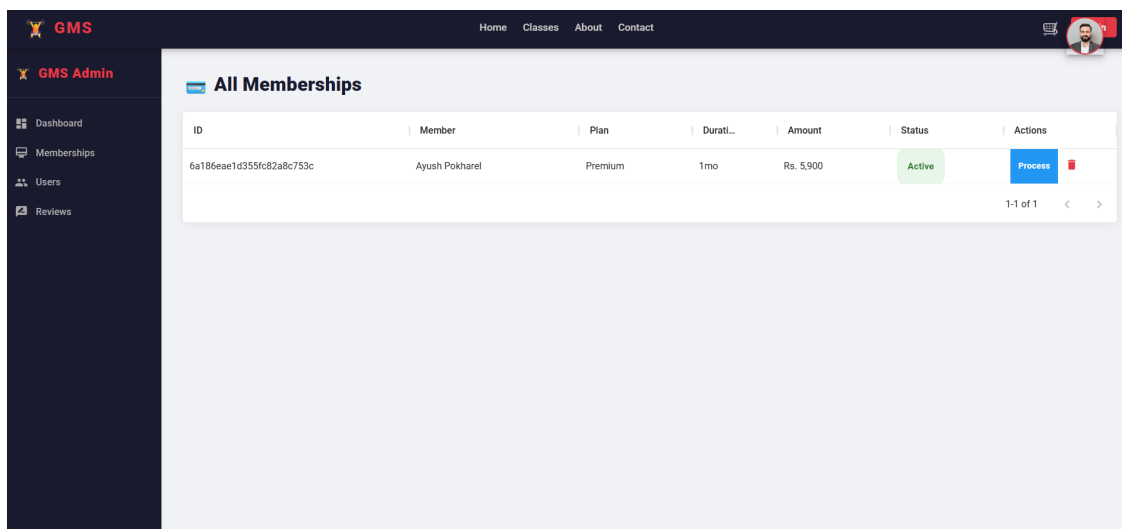


Figure 5.8: Membership Records – Admin View

5.8 Role-Based Access Control

Three distinct user roles – **Admin**, **Trainer**, and **Trainee** – have been implemented, each with dedicated dashboards and access permissions. This ensures data integrity and appropriate system usage at every level.

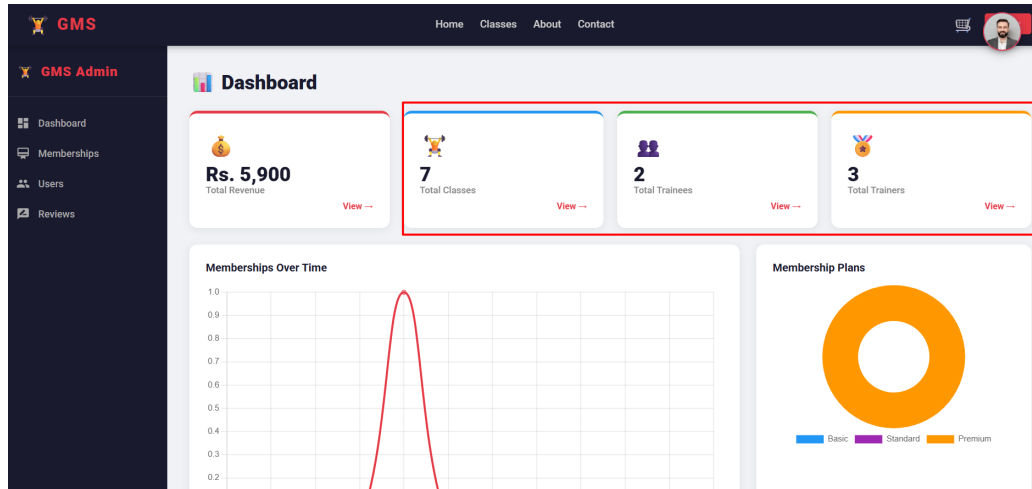


Figure 5.9: Role-Based Dashboards – Admin, Trainer, and Trainee Views

5.9 Class Reviews and Ratings

Trainees can submit reviews and star ratings for gym classes they have attended. Admins have the ability to moderate and delete reviews through the admin panel, ensuring quality feedback management.

The screenshot shows the GMS Member Reviews form. The top navigation bar includes 'Home', 'Classes', 'About', and 'Contact'. The main content area, titled 'Member Reviews', displays a review by Ayush Pokharel with a 5-star rating and the text 'It's the best'. Below this is a 'Cancel' button. The form itself includes a 5-star rating section, a text input field for 'Share your experience with this class...', and a 'Submit Review' button.

Figure 5.10: Class Reviews and Ratings System

Chapter 6

REFERENCES

- [1] Smith, J., & Taylor, R. (2020). Digital Transformation in Fitness Management: Integrating Modern Web Platforms. *Journal of Sports Technology*, 12(2), 45–58.
- [2] Jones, B., & Lee, C. (2021). The Evolution of Web Architectures: From Multi-page to Single-page Applications in GMS. *Web Engineering Review*, 14(3), 112–126.
- [3] Brown, T. (2019). NoSQL vs SQL: Choosing the Right Database for Scalable Web Applications. *Database Systems Journal*, 18(2), 23–35.
- [4] Davis, K. (2020). MERN Stack for Enterprise Solutions: Data Modeling with Mongoose. *Full-Stack Development Quarterly*, 7(4), 12–20.
- [5] Miller, S., & Kumar, A. (2023). Modern Web Security: JWT Authentication and Encrypted Data Persistence. *Journal of Cybersecurity*, 16(1), 77–90.
- [6] Johnson, M., & Adams, P. (2021). Third-party API Integration in Management Systems: Stripe and Cloudinary Implementations. *Fintech Innovations*, 9(1), 67–80.
- [7] Garcia, L., & Patel, S. (2023). The Future of AI and IoT in Health and Fitness Technology. *International Journal of Sports Science*, 15(4), 88–101.